AgriMarket

Project submitted to the

School of Engineering & Technology, BML Munjal University
for the partial fulfilment of the requirements to award the degree of

Bachelor of Technology
In

Computer Science and Engineering
School of Engineering and Technology

Submitted by

Bhavesh Gupta (240426)
Asmit Pandey(240413)
Parth Nagpal(240573)
Dhruv Aggarwal(240441)

BML MUNJAL UNIVERSITY
FROM HERE TO THE WORLD

Under the Guidance of
Dr. Nishtha Phutela

BML Munjal University
Kapriwas, Haryana– 122413
[May, 2026]

Certificate

Date: 04/05/2026

This is to certify that the work present in this Project entitled “AgriMarket” has been carried out by Bhavesh Gupta, Asmit Pandey, Parth Nagpal and Dhruv Aggarwal under my/our supervision. The work is genuine, original, and suitable for submission to the BML Munjal University for the award of Bachelor of Technology in School of Engineering and Technology.

Supervisor

Prof. / Dr. Faculty Name: Dr. Nishtha Phutela

Table of Contents

Sl.No.	Topic
1	Certificate
2	Table of Contents
3	Abstract
5	Introduction
6	Methodology
7	Discussion
8	Concluding Remarks
9	Societal Relevance
10	Future Scope

Abstract

In our project, we design a web-based centralized system, named AgriMarket, which aims at removing all intermediaries from the agribusiness supply chain process. Normally, farmers make use of inefficient, multi-level distribution processes where they earn less profit but customers have to pay more. We solve this problem through designing a platform that enables both farmers to upload their goods for sale and customers to buy their agricultural products without going through any intermediaries.

We used MySQL as a database management system, Flask for backend operations written in Python, and HTML, CSS, and Bootstrap for frontend development. Our system enables farmers to keep track of inventory and pricing information while buyers can look for and buy their required commodities. Our platform is designed in such a way that we can track total income generated along with logistical information. Our database consists of all the information about farmers and commodities in an efficient and normalized format.

Introduction

In this digital age, the management of sales processes in the agricultural industry continues to be fragmented, with significant dependence on middlemen and traditional physical markets. This method is complicated, opaque and manual, causing inconsistency in records, loss for farmers, and inefficiencies.

To remedy the above problems, a proposal for a system called AgriMarket has been made. The AgriMarket system will consist of a central database where all marketplace information will be incorporated. This will facilitate easy storage, retrieval and transaction processing in the market. Relational databases will be used where entities such as Farmers, Buyers, Products, Orders, Payments, Deliveries and relationships between these entities will be organized properly.

The system consists of various users such as Administrators, Farmers, and Buyers who have different responsibilities in the system. Administrators will oversee the operations in the market, track orders that have been placed, and report revenues generated. Farmers will manage their operations through the system by listing the inventory available and keeping updated on the harvests. Buyers will check the available inventory and place orders from the available inventory.

In terms of the database approach, the system has been designed through the application of ER modelling and adheres to normalization rules in order to minimize redundancy. The DBMS features that have been applied include composite primary keys, foreign key constraints, joins, aggregate operations, and transaction management through COMMIT and ROLLBACK commands.

The project not only highlights the practical implementation of DBMS concepts but also showcases how centralized data management can significantly improve the organization and execution of large-scale events.

Methodology

Design (ER Diagram, Schema, Normalization)

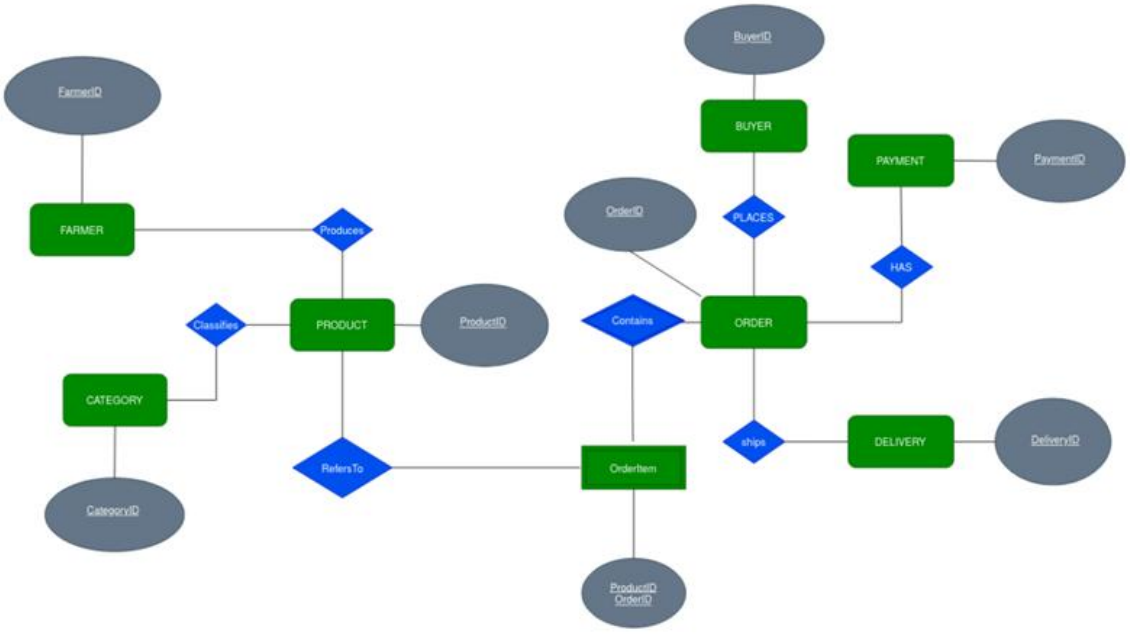
The development process for the AgriMarket was carried out in accordance with a structured method based on core DBMS principles. Firstly, key entities were defined, namely: Farmer, Buyer, Category, Product, Order, OrderItem, Payment, and Delivery.

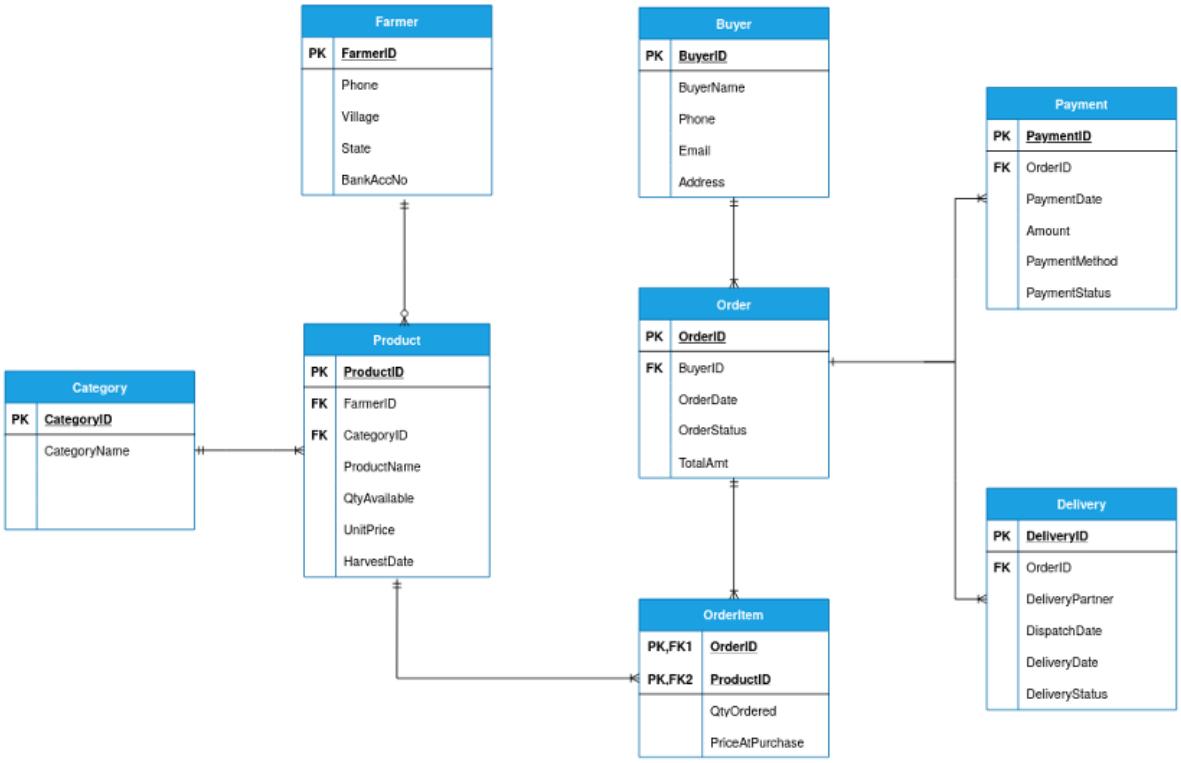
Product and Order can be considered central entities in the ER model. Attributes for each entity were defined, including: category of product, unit price, quantity in stock, and order status. The following relationships can be found within the ER model:

- One Farmer lists multiple Products (1:N).
- One Buyer places multiple Orders (1:N).
- One Order contains multiple Products, handled via the OrderItem bridge table (M:N).
- One Order has exactly one Payment and one Delivery record (1:1).

Normalization was applied to organize the data efficiently and remove redundancy:

- 1NF: All attributes are atomic (no repeating groups).
- 2NF: Partial dependencies were removed. In the OrderItem table, non-key attributes (QtyOrdered, PriceAtPurchase) depend entirely on the composite primary key (OrderID, ProductID).
- 3NF: Transitive dependencies were eliminated by separating related data into different tables (e.g., storing Category details separately instead of repeating them in the Products table).


FarmerID
FARMER
Produces
PRODUCT
ProductID
Classifies
CATEGORY
CategoryID
RefersTo
OrderItem
ProductID
OrderID
Contains
ORDER
OrderID
PLACES
BUYER
BuyerID
PAYMENT
PaymentID
HAS
ships
DELIVERY
DeliveryID


Farmer
PK FarmerID
Phone
Village
State
BankAccNo
Buyer
PK BuyerID
BuyerName
Phone
Email
Address
Payment
PK PaymentID
FK OrderID
PaymentDate
Amount
PaymentMethod
PaymentStatus
Category
PK CategoryID
CategoryName
Product
PK ProductID
FK FarmerID
FK CategoryID
ProductName
QtyAvailable
UnitPrice
HarvestDate
Order
PK OrderID
FK BuyerID
OrderDate
OrderStatus
TotalAmt
Delivery
PK DeliveryID
FK OrderID
DeliveryPartner
DispatchDate
DeliveryDate
DeliveryStatus
OrderItem
PK,FK1 OrderID
PK,FK2 ProductID
QtyOrdered
PriceAtPurchase

Implementation

The implementation of the system follows a full-stack architecture using Python (Flask) for the backend, MySQL for the database, and HTML/CSS/Bootstrap for the frontend.

- Backend: The backend deals with all server-side programming logic. It connects to the MySQL database by using the library named mysql.connector. The application employs a modular design approach where there are separate routes depending on the user's role (for example, "/admin", "/farmer", "/buyer"). Database transactions are widely used; during order placement, the backend writes an order, reduces the stock, and makes deliveries/payments concurrently within a try-except-finally block..

- Frontend: he frontend offers an easy-to-use GUI through a current layout of the web application. Data is retrieved from the backend and displayed in tables and product cards with the help of templates.

- Functional Implementation

  The AgriMarket backend system utilizes Flask framework to interact with MySQL database through the use of mysql.connector package. In regards to authentication and role-based permissions, the AgriMarket system uses the session property of Flask to determine what dashboard each user should be routed to, depending on whether they have admin, farmer or buyer roles.

  The functionalities of AgriMarket are categorized in various operating routes:

- Farmer Dashboard (/farmer): Allows farmers to register new inventories whereby an INSERT command is executed against the Product Table containing category, quantity and pricing. Additionally, farmers can monitor their orders and mark them as shipped, where an update operation will be performed for OrderStatus and DeliveryStatus with ‘shipped’.

- Buyer Dashboard (/buyer): Allows buyers to view live inventories through dynamic SQL commands filtered by product name, category and sorted by price. They can also manage orders and cancel them whereby any deductions made from Product Table are returned back and payment can be completed and dynamically set the PaymentStatus to completed unless ‘pay later’ is checked.

- Final, the Admin Dashboard (/admin) employs sophisticated and dynamically built SQL queries for creating platform-wide reports. For example, the /sales route produces a revenue report using the query SUM(oi.QtyOrdered * oi.PriceAtPurchase) through JOINs on the Farmer, Product, OrderItem, and Order tables. The server-side application dynamically adds WHERE, GROUP BY f.FarmerID, and HAVING conditions depending on the admin's filter criteria (date ranges and minimum revenue amounts). To guarantee complete transparency during the demonstration, the server-side application also has an /api/query_details route that records and forwards the actual SQL statement and its runtime to the frontend interface.

Algorithm

Algorithm 1: Role-Based Authentication Workflow

Input: User Role (admin, farmer, or buyer), Identifier (Phone Number or Password). Output: Active session and redirection to the appropriate dashboard.

1. Start: User submits the login form.
2. Check Role: Determine if the requested role is Admin, Farmer, or Buyer.

3. If Admin:
    - Verify if the entered password matches the system admin credentials.
    - If valid, initialize session['role'] = 'admin' and redirect to the Admin Dashboard.
    - If invalid, display an error message and reload the login page.

4. If Farmer / Buyer:
    - Execute a SELECT query on the respective table (Farmer or Buyer) where Phone = identifier.
    - If a record is found, extract the UserID and UserName.
    - Initialize the Flask session with these credentials and redirect to the respective portal.
    - If no record is found, display an "Invalid Credentials" error.

5. End.

Algorithm 2: Farmer Inventory Addition

Input: product_name, qty, price, category_id.
Output: New product record stored in the database.

1. Start: Farmer submits the "Add Inventory" form.
2. Retrieve Session: Fetch the FarmerID from the active secure session.
3. Execute Query: Run an INSERT INTO Product SQL command using the provided inputs and the session's FarmerID.
4. Commit: Save the changes permanently to the database.
5. Refresh: Reload the Farmer dashboard to display the updated inventory list.
6. End.

Algorithm 3: Buyer Order Placement (ACID Transaction)

Input: BuyerID (from session), product_id, qty_to_buy.
Output: Completed order with updated stock, or a rolled-back transaction on failure.

1. Start: Buyer clicks "Buy" on a product.
2. Stock Verification: Execute a SELECT query to check the QtyAvailable and UnitPrice for

the requested product_id.

3. Condition Check: If QtyAvailable < qty_to_buy, halt the process and display a "Not enough stock" error.
4. Initialize Transaction: If stock is sufficient, calculate TotalAmt = qty_to_buy * UnitPrice and begin the database transaction (conn.cursor()).

5. Try Block Execution:
    - Step A: INSERT INTO Order to log the purchase and retrieve the new lastrowid (order_id).
    - Step B: INSERT INTO OrderItem using the new order_id, product_id, and exact price.
    - Step C: UPDATE Product SET QtyAvailable = QtyAvailable - qty_to_buy to deduct the stock.
    - Step D: INSERT INTO Delivery initializing the status to 'Processing'.
    - Step E: INSERT INTO Payment initializing the status to 'Pending'.

6. Commit/Rollback Logic:
    - If all 5 steps succeed, execute conn.commit() to permanently save the transaction.
    - If any database exception occurs during the steps, execute conn.rollback() to cancel all changes and prevent data anomalies.
7. End.

Algorithm 4: Admin Dynamic Report Generation

Input: Optional filters (Farmer Name, Date Range, Minimum Sales, Maximum Sales). Output: Aggregated revenue dataset.

1. Start: Admin accesses the /sales route.

2. Construct Base Query: Initialize the SELECT query with required JOIN operations linking the Farmer, Product, OrderItem, and Order tables.

3. Apply WHERE Filters: Check if the Admin provided a specific Farmer Name or Date Range. If yes, dynamically append these conditions to the WHERE clause.

4. Aggregate Data: Append GROUP BY f.FarmerID to calculate the total sales per farmer using SUM(oi.QtyOrdered * oi.PriceAtPurchase).

5. Apply HAVING Filters: Check if the Admin set Minimum or Maximum sales thresholds. If yes, dynamically append these to a HAVING clause.

6. Execute & Render: Run the finalized query, fetch the aggregated data, and pass it to the

frontend templating engine to render the report table.

7. End.

Discussion/Results/Observations

1. Discussion

This project also demonstrates how database management systems and web technologies can be used practically in solving real-life problems. For instance, the AgriMarket has been able to incorporate various roles including farmers, buyers, and administrators into a single platform allowing for easy coordination and communication between all participants in the agricultural market.

In terms of functionality, the use of a relational database system was quite effective when dealing with complex datasets including products, orders, payments, and delivery. In particular, the use of foreign keys and joining allowed the system to retrieve interrelated records from different databases efficiently. Thus, the power of structured databases has been proven while handling transactional applications.

In terms of usability, the developed software solution provides users with a simple interface through which they can easily list their products, check available items, and manage their orders. This characteristic is particularly relevant in the field of agriculture where users may lack some technical knowledge.

In conclusion, it should be noted that the presented discussion proves that combining normalized database design, transaction management, and web technologies leads to creating a functional system which is scalable enough to undergo further improvement in the field of agricultural supply chains.

2. Results

The AgriMarket system was deployed and tested in terms of all functions, namely registration, creation of listings, placing orders, processing payments, and delivering products. The system efficiently processes information flows related to communication among farmers, buyers, and administrators. The system also executes complicated queries involving many tables (e.g., placing an order and selling).

Additionally, the system processes transactions appropriately, which allows maintaining data integrity and consistency while executing tasks, such as ordering and canceling orders. The user interface is user-friendly, providing users with opportunities to easily navigate and interact with the system.

```
245 ●  SELECT * From farmer
```

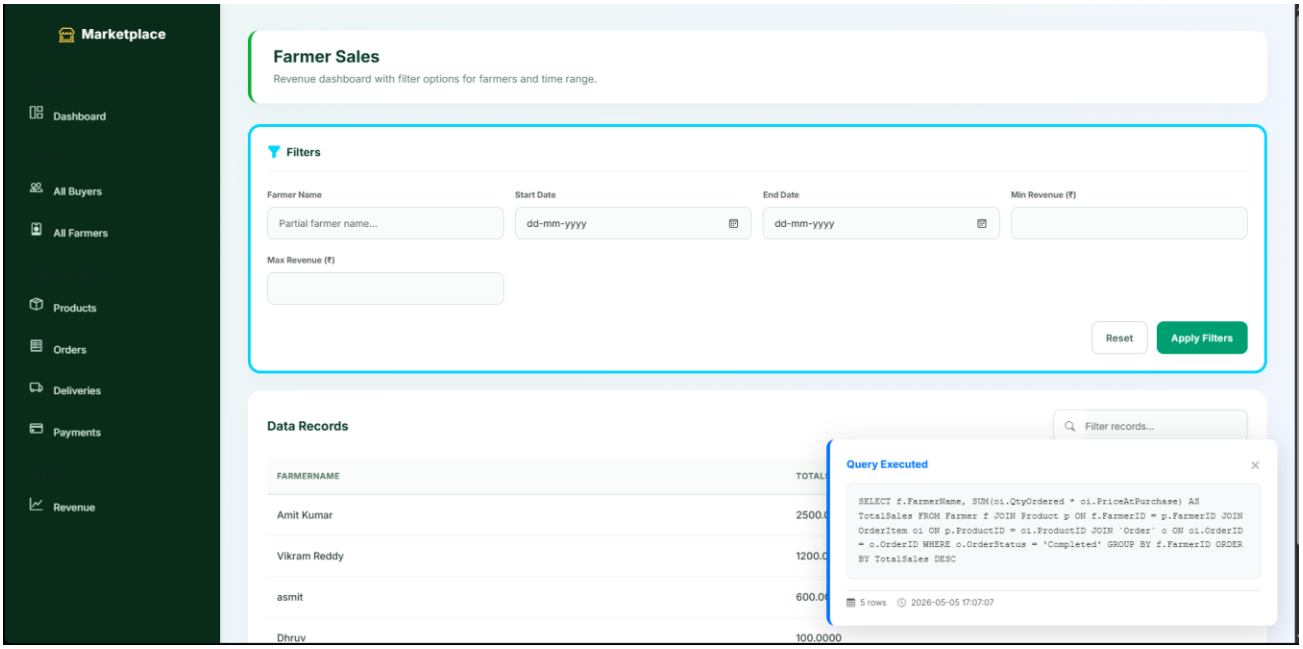
Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

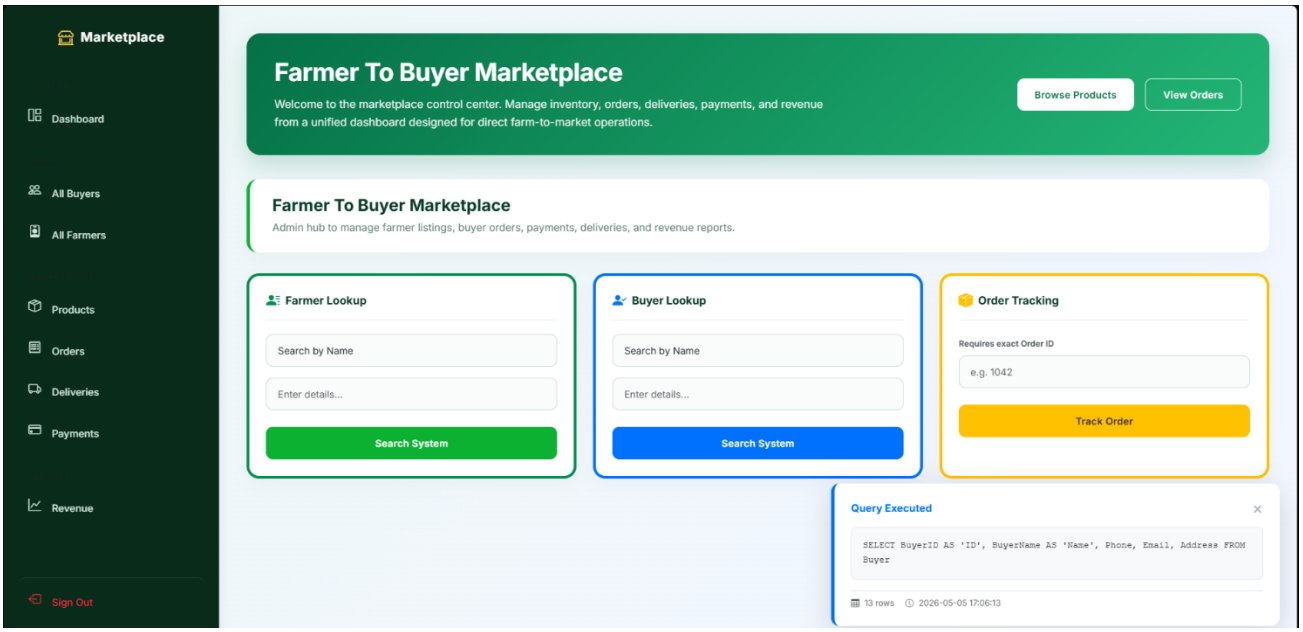
FarmerID	FarmerName	Phone	Village	State	BankAccNo
s	Amit Kumar	9876543210	Kharak	Haryana	SBIN0001111
2	Rajesh Singh	9876543211	Palampur	Himachal Pradesh	HDFC0002222
3	Suresh Bhai	9876543212	Anand	Gujarat	ICIC0003333
4	Ramesh Patil	9876543213	Nashik	Maharashtra	UTIB0004444
5	Manoj Gowda	9876543214	Mandya	Karnataka	SBIN0005555
6	Vikram Reddy	9876543215	Warangal	Telangana	HDFC0006666
7	Anil Sharma	9876543216	Mathura	Uttar Pradesh	ICIC0007777
8	Karan Yadav	9876543217	Patna	Bihar	UTIB0008888
9	Dhruv	9999999999	Alwar	Rajasthan	1234356778623
10	Dhruv	9084735571	bulandshahr	uttarpradesh	9999999111111111
11	asmit	9999639411	gurgaon	haryana	1234567
NULL	NULL	NULL	NULL	NULL	NULL

```
245 ●  SELECT * From `Order`
246
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

OrderID	BuyerID	OrderDate	OrderStatus	TotalAmt
1	1	2026-04-16 10:00:00	Completed	2500.00
2	2	2026-04-16 11:30:00	Processing	6000.00
3	3	2026-04-16 14:15:00	Pending	500.00
4	4	2026-04-17 09:00:00	Shipped	2400.00
5	5	2026-04-17 09:45:00	Completed	1200.00
6	6	2026-04-17 10:30:00	Pending	3000.00
7	7	2026-04-17 11:00:00	Processing	1000.00
8	8	2026-04-17 12:15:00	Pending	4500.00
9	9	2026-04-18 02:34:21	Completed	50.00
10	9	2026-04-18 08:30:57	Completed	50.00
11	11	2026-04-20 10:21:50	Completed	10.00
12	11	2026-04-20 10:24:18	Cancelled	20.00
13	11	2026-04-20 10:24:26	Cancelled	10500.00
14	11	2026-04-20 10:24:38	Cancelled	59880.00
15	9	2026-04-24 12:25:42	Completed	600.00
16	12	2026-05-04 15:52:49	Cancelled	75.00
17	9	2026-05-04 16:28:09	Pending	50.00
18	13	2026-05-05 15:15:59	Shipped	540.00


Marketplace
Dashboard
All Buyers
All Farmers
Products
Orders
Deliveries
Payments
Revenue
Farmer Sales
Revenue dashboard with filter options for farmers and time range.
Filters
Farmer Name
Partial farmer name...
Start Date
dd-mm-yyyy
End Date
dd-mm-yyyy
Min Revenue (₹)
Max Revenue (₹)
Reset
Apply Filters
Data Records
Filter records...
FARMERNAME
TOTAL
Amit Kumar
2500.0
Vikram Reddy
1200.0
asmit
600.0
Dhruv
100.0000
Query Executed
SELECT f.FarmerName, SUM(oi.QtyOrdered * oi.PriceAtPurchase) AS TotalSales FROM Farmer f JOIN Product p ON f.FarmerID = p.FarmerID JOIN OrderItem oi ON p.ProductID = oi.ProductID JOIN `Order` o ON oi.OrderID = o.OrderID WHERE o.OrderStatus = 'Completed' GROUP BY f.FarmerID ORDER BY TotalSales DESC
5 rows
2026-05-05 17:07:07


Marketplace
Dashboard
All Buyers
All Farmers
Products
Orders
Deliveries
Payments
Revenue
Sign Out
Farmer To Buyer Marketplace
Welcome to the marketplace control center. Manage inventory, orders, deliveries, payments, and revenue from a unified dashboard designed for direct farm-to-market operations.
Browse Products
View Orders
Farmer To Buyer Marketplace
Admin hub to manage farmer listings, buyer orders, payments, deliveries, and revenue reports.
Farmer Lookup
Search by Name
Enter details...
Search System
Buyer Lookup
Search by Name
Enter details...
Search System
Order Tracking
Requires exact Order ID
e.g. 1042
Track Order
Query Executed
SELECT BuyerID AS 'ID', BuyerName AS 'Name', Phone, Email, Address FROM Buyer
13 rows
2026-05-05 17:06:13

3. Observations

While developing and testing the AgriMarket application, it was noted that the presence of a well-designed and normalized database greatly enhances data integrity and decreases data redundancy.

Foreign keys and joins helped to fetch relevant data from various tables, particularly for complicated processes such as order tracing. The transaction control process helped to perform complex processes in a flawless manner without any database inconsistency issues.

Moreover, it was noted that role-based access control not only helps to secure a system but also makes it easier to use by providing certain features to particular individuals. Combining a user-friendly web interface with backend code made it easy for users to interact with the system.

Concluding Remarks

AgriMarket serves as an exemplary case study illustrating the implementation of database management systems and web applications to solve practical issues encountered by farmers. Through the removal of intermediaries and direct communication between growers and customers, the application is able to provide better transparency, efficiency, and revenue generation in the supply chain.

The project incorporates several fundamental components of DBMS, including normalization, relational modeling, and transactions. The system has a great deal of potential to transform itself into a robust commercial-grade software platform through additional features, including cloud hosting, security measures, and payments.

Societal Relevance

There are multiple reasons why the AgriMarket system is societally important since it tackles important problems that the agriculture industry currently faces. For instance, the system helps to decrease the number of middlemen, ensuring that farmers get adequate compensation for their crops. Consequently, both farmers and consumers can benefit from the system as the latter will pay less due to reduced costs while farmers' earnings will increase.

Furthermore, the system helps to introduce technology to farmers, thus bridging the gap between producers and consumers, who may live in different parts of the country. Moreover, by making the processes of trading agricultural produce easier and more efficient, the AgriMarket system helps to decrease food wastage in the country

Future Scope

AgriMarket system can be further refined in many aspects, thus making it highly scalable and practical. For instance, incorporation of secure payment systems like UPI or card-based payments will make it much easier to conduct the transaction. Using this system through cloud services would greatly enhance its usability as well as improve its scalability.

More sophisticated functionalities like real time pricing prediction, demand forecasting, recommendation engines, among others, based on machine learning can be integrated into the system to facilitate decision making by both farmers and customers. Developing mobile apps for this system would provide more access, especially to people living in the rural areas.

Contribution of Group Members

Bhavesh Gupta (240426):
Implemented core backend functionality with the help of Flask, including user authentication for admin, farmer and buyer as well as integration of queries with application logic.

Asmit Pandey (240413):
Did ER modelling along with normalizing the database up to 3NF level as well as creation of relational tables for different purposes with proper relationships defined.

Parth Nagpal (240573)
Created major functional modules of the application such as management of products, order placement process and OrderItem relationship management along with efficient queries.

Dhruv Aggarwal (240441)
Implemented transactional functionality such as order placement, updating payments, and delivery management with the use of commits and rollbacks in order to maintain consistency in data.

Acknowledgment of AI Utilization

In this project, artificial intelligence (AI) was used to provide assistance during various development processes.
The uses of AI include the following:

Explanation and clarification of DBMS concepts like normalization, ER modeling, and transaction processing
Assistance in organizing and improving report sections, such as Introduction, Methodology, and Discussion
Code debugging in Flask and SQL queries
Advice on best practices in web development and systems engineering
All the AI-generated material has been checked, adjusted, and verified by the team before being incorporated into the project. Thus, the results presented represent the independent knowledge and work of the team.
I hereby confirm that AI technologies were used only as an aid and the resulting project is the team's original creation.